

RAI-853: Advanced Robotics with ROS

Lec 06 - Robot Simulation in Gazebo

Instructor:

Dr. Khawaja Fahad Iqbal

Deputy HoD (Department of Robotics & AI, SMME, NUST)

Scientific Director (Intelligent Robotics Lab, NCAI)



- **Starting Gazebo**
- **Inertia Tags**
- **Collision Tags**
- **Robot Simulation in Gazebo**
- **Gazebo Communication**

- **Starting Gazebo**
- Inertia Tags
- Collision Tags
- Robot Simulation in Gazebo
- Gazebo Communication

- ROS 2 in a virtual machine (VM) works well for **robot modelling and visualization** even when running RViz.
- However, **with Gazebo**, chances are that the VM won't be enough. VirtualBox doesn't work well with 3D simulation tools.
- Therefore, for simulation in Gazebo, it is recommended that you have Ubuntu installed with a **dual boot**.

- Gazebo is a 3D simulation engine. It contains a **physics engine** (with gravity, friction, and other physical constraints) with which you can simulate a robot, just like if it were in the real world.
- You can develop your application using mostly the Gazebo simulation and then work with the real robot.
- This brings a lot of benefits. For example, you can work on robots that don't exist yet, test extreme use cases without damaging the real robot, create custom environments you can't access on a daily basis, work remotely, and so on.

- RViz is a 3D **visualization tool**. With RViz, you can visualize the URDF, the TFs, and the data you get from ROS topics. This is a great tool to use when developing, as you can check that what you are doing is correct.
- **RViz is NOT a simulation tool**. You don't simulate anything; you only visualize what already exists. So, the robot and all the data you see in RViz is only the representation of what's happening externally (of RViz).
- Instead of seeing and interacting with all the data in the terminal, you can do that with a **graphical interface** and see the data in 3D. That's (very simply put) what RViz brings to you.

- Conversely, Gazebo is a **simulation tool**. It will simulate **gravity and the real physical properties** of the robot.
- It also has some **control plugins** so that you can simulate the hardware control, and even publish the joint states and TFs for your robot.
- Ultimately, it's not a competition between Gazebo and RViz; both are complementary.

- Gazebo is available if you don't have a real robot, if you don't want to use it, or if you want to test a robotics system in a different environment.
- With Gazebo, you can replicate the behavior of your robot, and make it very close to what it would do in real life.
- On top of Gazebo, you can use RViz to visualize the TFs and other important data from your application.
- RViz is still a very useful tool to have during the development phase. So, the dilemma is not Gazebo versus RViz, but instead, **real robot versus Gazebo**.

- First, you need to install Gazebo. To get everything we need for Gazebo, install this additional package:

```
sudo apt install ros-<distro>-ros-gz
```

- Make sure to replace <distro> with your current ROS 2 distribution, and after running this command, source your environment.
- Since we are using Ubuntu 24.04, this will install **Gazebo Harmonic**.

- It is important to note that **Gazebo is actually independent from ROS 2**. You can run Gazebo on its own, without ROS 2. In fact, you could even install Gazebo without any ROS 2 packages.
- We will start by running Gazebo on its own. Then, we will see how to connect Gazebo and ROS.
- So, to start Gazebo (without ROS), run this command in a terminal:

gz sim

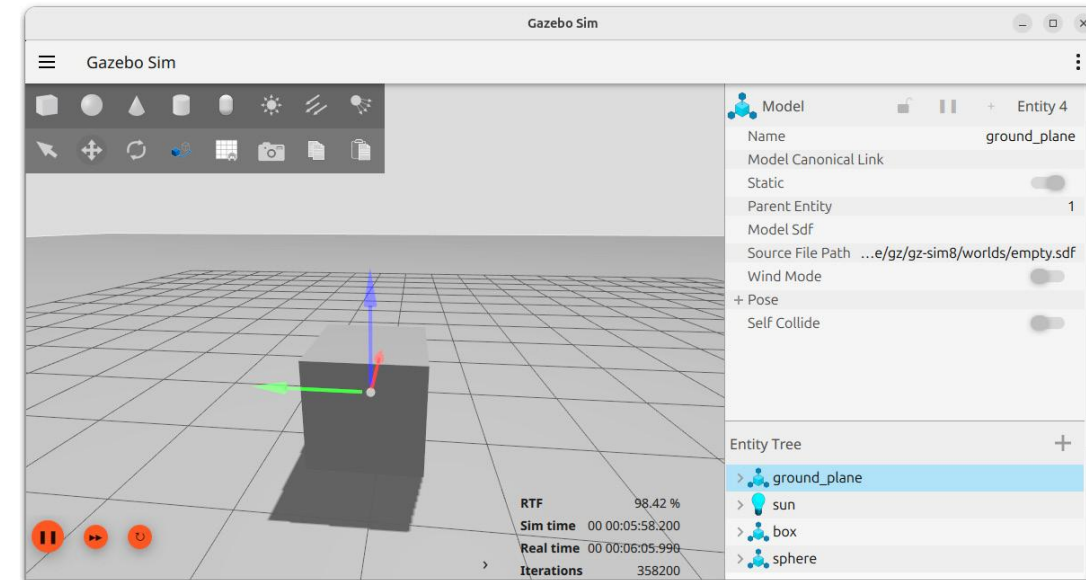
- You will be taken to a Gazebo quick-start menu. There, you can click on Empty to **load an empty world**. You also have other existing worlds (click on the different images, or use the search bar) that you can explore later on your own.
- To stop the simulation, press Ctrl + C in the terminal where you started Gazebo.
- When running the Gazebo command, you can also **directly specify the world** you want to launch.
- World description files in Gazebo use the SDF format (a file with a .sdf extension), which is quite similar to URDF. Let's start Gazebo with the empty world:

gz sim empty.sdf

- Now, take the time to move around the space using mouse controls.
- At the **bottom left of the screen**, you will see a play button. Click on it to start the simulation.
- After starting the simulation, you will see a percentage at the bottom-right part of the screen, representing the **real-time factor**. Basically, this will tell you immediately whether your computer is powerful enough for Gazebo.

- For example, if you have a real-time factor of about 98%, which means that the simulated time in Gazebo keeps up with real time at 98% speed (i.e., after 100 real seconds, 98 seconds have passed in Gazebo).
- If your percentage is too low, it may be a sign that you need better performance.

- Let's return to the simulation we started—at the top of the screen, you can click on the different shapes and add them to the empty space. Take some time to experiment with this. Add a box into the space.
- Find the translation mode and rotation mode. Move the box around (especially on the z axis) and see what happens. If you lift the box up, then you should see the box falling down on the floor. This is because of the **gravity and physical properties** of the box.



- What we can do next is explore the communications used by Gazebo. Gazebo also uses topics and services, but those are not the same as ROS 2 topics and services.
- For example, you can list all Gazebo topics with this command:

gz topic -l
- You will see a lot of topics, but if you try to run **ros2 topic list** in another terminal, **none of those topics will appear**. With this, you can see that Gazebo is completely independent of ROS 2, and they do not interact with each other.

- Starting Gazebo
- **Inertia Tags**
- Collision Tags
- Robot Simulation in Gazebo
- Gazebo Communication

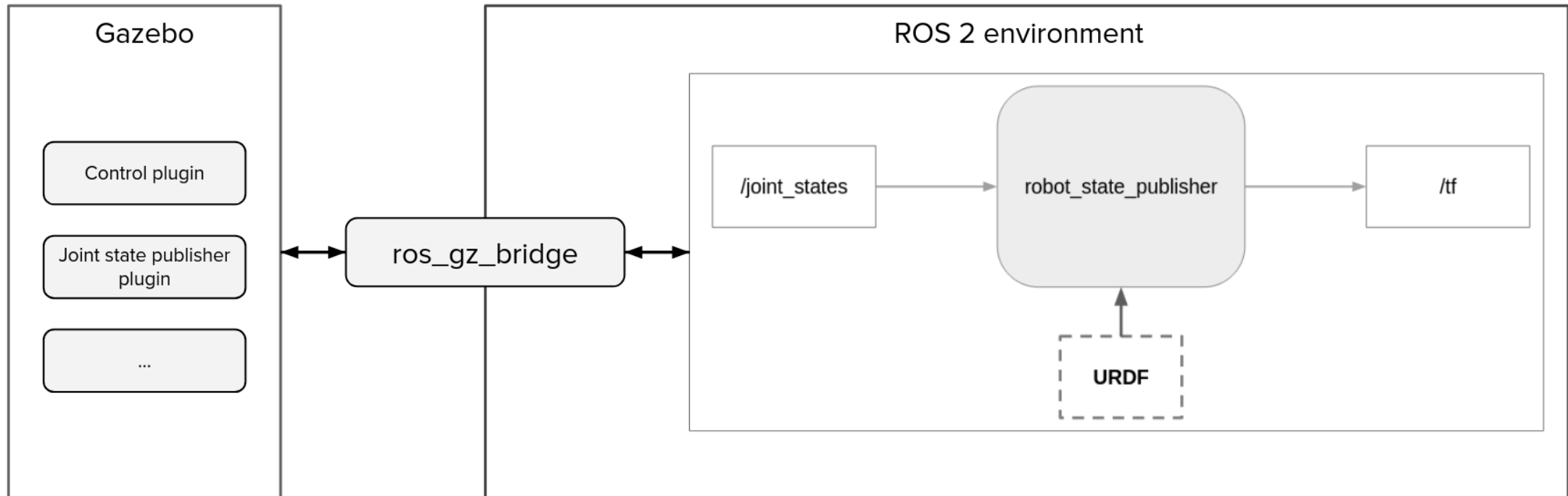
- First of all, you can **start Gazebo using a ROS 2 launch file** from the `ros_gz_sim` package. This will be more practical for us because when we write our own launch file, we can include this one:

```
ros2 launch ros_gz_sim gz_sim.launch.py
```

- This will start Gazebo the same way we did with the `gz sim` command. You can also specify the world to launch with the `gz_args` argument:

```
ros2 launch ros_gz_sim gz_sim.launch.py  
gz_args:=empty.sdf
```

- However, even if we started Gazebo from a ROS 2 launch file, **Gazebo is still independent**. Try to list all the nodes and topics in a terminal; you will see the same result as before.
- To connect Gazebo and ROS 2 topics (or services), you **need to create a bridge** between them. The **ros_gz_bridge** package does that for us.
- We will only need to provide some configuration to specify which topics we want to bridge.



- On the right, we have our current ROS 2 application with the `robot_state_publisher` node, publishing on the `/tf` topic.
- On the left, we have Gazebo. Inside Gazebo, we will add **plugins (also called systems)** to simulate the hardware behavior of the robot. For example, you could have one plugin to control the two wheels, and one plugin to publish the joint states for the wheels.
- Then, to make everything work together, we will use the `ros_gz_bridge` package. With the joint state example, the Gazebo Joint state publisher plugin will publish the joint states with a Gazebo topic. Using `ros_gz_bridge`, we will match this topic with the ROS 2 `/joint_states` topic.

- Your ROS 2 application will be the same, whether you work on a Gazebo simulation or a real robot.
- If you work on a **real robot**, then you would directly control the wheels and get the joint state data from encoders.
- With **Gazebo**, you use plugins to simulate the hardware.

- Here are the steps we will take in the following sections to create the Gazebo simulation for our robot:
1. Adapt the URDF for Gazebo. For a robot to work on Gazebo, we first need to provide **inertial and collision properties** in the URDF.
 2. Once the URDF is correct, we will start Gazebo and spawn the URDF in it. At this point, we will also **create a package with a launch file**.
 3. We will then **add some plugins (systems)** to control the robot, using the `ros_gz_bridge` package to make those plugins communicate with our ROS 2 application.

- We could try to spawn our robot directly into Gazebo, but it won't work, as the URDF is missing two key elements—**inertial and collision properties**. Gazebo needs those to correctly simulate a robot.
- Thus, before we do anything with Gazebo, we need to come back to our URDF and add those properties.
- For each link of the robot that represents a physical part, we will add an <inertial> tag and a <collision> tag.

- An <inertial> tag will contain a few elements, including a 3x3 matrix representing an inertia tensor.
- In this workshop, we won't dive into the theoretical inertia details.
- Instead, we will focus on finding the correct formulas and applying them, allowing us to spawn the robot and quickly move on to the next steps.

- Currently, our URDF is split into three files. We will add some code to those files:
1. **common_properties.xacro:** Here, we will add some macros to specify the inertial properties for a box, a cylinder, and a sphere. This way, we only need to write the inertial formulas once for those shapes, and you can reuse them in any of your projects.
 2. **mobile_robot.xacro:** Inside each link representing a physical part, we will use the corresponding inertial macro we defined previously.
 3. **my_robot.urdf.xacro:** Nothing changes here; we still import the two previous files.

- We will create three Xacro macros containing an `<inertial>` tag—one for a box, one for a cylinder, and one for a sphere.
- Inside a URDF `<inertial>` tag, you will need to provide:
 1. The mass of the element (in kg).
 2. The origin for the inertia (in meters and radians).
 3. The nine elements of the inertia tensor, or matrix (in kg per sqm). Since the matrix is symmetrical, we only need six elements—`ixx`, `ixy`, `ixz`, `iyy`, `iyz`, and `izz` (for example, `ixy` and `iyx` are the same, so we omit the second one).

- If you are designing your robot with **CAD software**—for example, with SolidWorks—then you can export each property directly from the software and add them to your URDF. As we don't have this software, we will need to make the computation ourselves.
- We will arbitrarily decide on a mass property for each link—while, of course, trying to have values that make sense.
- Now, how do we compute the inertia matrix? This is usually the hardest part when writing the <inertial> tags.

- The first macro will be for a **box inertia**. If you go through the mechanics literature, things could be a bit confusing, as they use width, depth, and height (w, d, and h).
- In ROS 2, we have specified the length, width, and height for the x, y, and z dimensions. Which one corresponds to which?
- One easy way to write this matrix properly is to realize that **one component for one dimension uses the two other dimensions**.

- For example, to compute the w component (i_{xx} in the matrix), we use $h(z)$ and $d(y)$.
- Only following this can remove a lot of confusion, especially with the different naming conventions.
- Here is what we will use (on the left, the Mechanics literature value, and on the right, the ROS 2 value):
 1. **w : the x dimension**
 2. **d : the y dimension**
 3. **h : the z dimension** (this is also the axis pointing up)

- Here is the `<inertial>` tag for a **box**. You can **add this after the `<material>` tags**, inside the `<robot>` tag:

```
<xacro:macro name="box_inertia" params="m x y z o_xyz
o_rpy">
  <inertial>
    <mass value="{m}" />
    <origin xyz="{o_xyz}" rpy="{o_rpy}" />
    <inertia ixx="{(m/12) * (z*z + y*y)}" ixy="0" ixz="0"
iyy="{(m/12) * (x*x + z*z)}" iyz="0" izz="{(m/12) * (x*x
+ y*y)}" />
  </inertial>
</xacro:macro>
```

- Let's now write the **macro for a cylinder**. This one is a bit easier. We have two components—radius and height. This will correspond to the radius and length we defined in the URDF:

```
<xacro:macro name="cylinder_inertia" params="m r l o_xyz
o_rpy">
  <inertial>
    <mass value="{m}" />
    <origin xyz="{o_xyz}" rpy="{o_rpy}" />
    <inertia ixx="{(m/12) * (3*r*r + l*l)}" ixy="0" ixz="0"
iyy="{(m/12) * (3*r*r + l*l)}" iyz="0" izz="{(m/2) * (r*r)}" />
  </inertial>
</xacro:macro>
```

- Finally, we can write the **macro for a (solid) sphere**. This is the easiest, and we only have one component —the sphere radius:

```
<xacro:macro name="sphere_inertia" params="m r o_xyz
o_rpy">
  <inertial>
    <mass value="\${m}" />
    <origin xyz="\${o_xyz}" rpy="\${o_rpy}" />
    <inertia ixx="\${(2/5) * m * r * r}" ixy="0" ixz="0"
iyy="\${(2/5) * m * r * r}" iyz="0" izz="\${(2/5) * m * r * r}" />
  </inertial>
</xacro:macro>
```


- We can now use those macros to provide the inertial property for each link of the robot.
- As the **base_footprint** doesn't represent a physical part (it's what we call a virtual link), it **won't have an inertia**.
- For all the other links (base, right wheel, left wheel, and sphere), we will use the inertial macros.
- Open the **mobile_base.xacro** file, which is where we will continue with the code.

- Now, to add the inertial property for a link, you need to add an `<inertial>` tag inside the `<link>` tag. To add this tag, we will use the macros we previously created.
- Let's start with **base_link**. Inside the `<link name="base_link"></link>` tag, and **after the `<visual>` tag**, add the `box_inertia` macro:

```
<xacro:box_inertia m="5.0" x="${base_length}"
y="${base_width}" z="${base_height}" o_xyz="0 0
${base_height / 2.0}" o_rpy="0 0 0" />
```

- We specify all the parameters required for the macro:
 1. **Mass:** Here, we decided that the box would be 5.0 kg.
 2. **Box properties:** The x, y, and z dimensions (we have created the macro so that we can use the ROS 2 axis system convention).
 3. **Inertia origin:** The inertia is related to the box itself, so if the box has an offset relative to the joint origin, you need to take this offset into account. Basically, you can use the same values you wrote in the visual origin.

- Now, **add the inertia for the two wheels.**
- You will add the `cylinder_inertia` macro **inside the `wheel_link` macro** (it's perfectly fine to use a macro within another macro).
- Make sure to place it inside the `<link>` tag and after the `<visual>` tag. This inertia macro will apply to both wheels:

```
<xacro:cylinder_inertia m="1.0"
r="${wheel_radius}" l="${wheel_length}" o_xyz="0 0
0" o_rpy="${pi / 2.0} 0 0" />
```

- Here are the parameters we specify:
 1. **Mass:** Set the mass for each wheel to 1.0 kg.
 2. **Cylinder properties:** The radius and length of the cylinder.
 3. **Inertia origin:** The visual is centered around the link origin, so we don't need to add any offset.

- Finally, we add the **sphere_inertia** macro for **caster_wheel_link**:

```
<xacro:sphere_inertia m="0.5" r="{wheel_radius / 2.0}" o_xyz="0 0 0" o_rpy="0 0 0" />
```

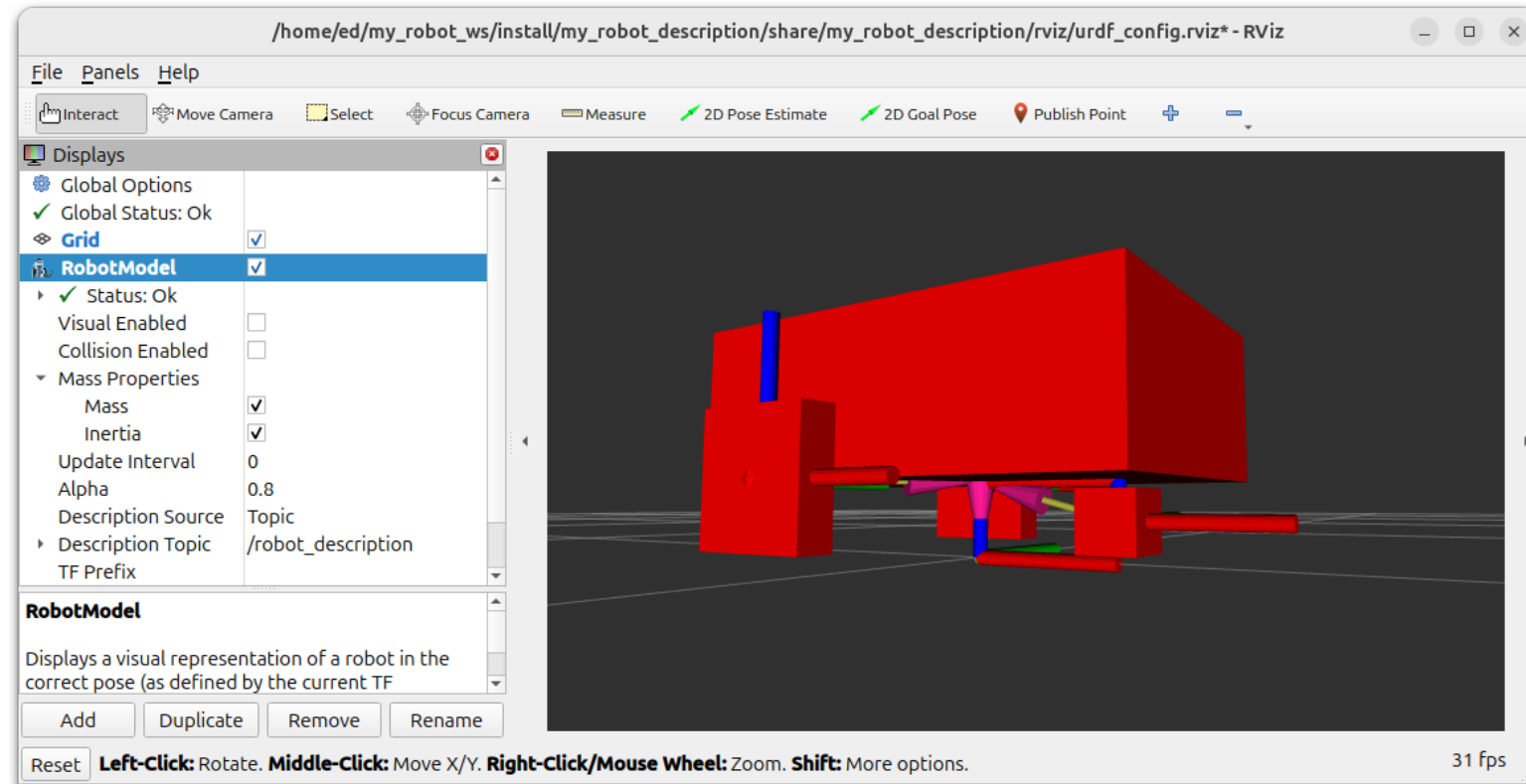
- We choose 0.5 kg for the mass, and then we provide radius, which is the only sphere property. We don't need to specify any offset for the origin.
- That's it for the `<inertial>` tags. Let's now verify that we provided the correct dimension values and origins, using RViz.

- To make sure that the inertial property is correct for each link, you can use RViz.
- As we already have created a launch file inside the my_robot_description package, let's use it to visualize the URDF in RViz.
- **Don't forget to build and source the workspace** before you start RViz. Then, start the launch file:

colcon build
source install/setup.bash

ros2 launch my_robot_description display.launch.xml

- You will see the same view and configuration as before.
- To see the inertia, first **disable the visual**. On the left menu, open RobotModel and uncheck Visual Enabled.
- Then, still inside RobotModel, open Mass Properties and **check the Inertia box**. You should see something like this:



- With this view, you can **easily spot errors**. For example, if the offset for the `base_link` inertia is not right, then the box will not be correctly placed.
- Another **common error will be in the rotation of the wheel inertia**. In the preceding figure, you can see that the inertia box is more or less on top of the wheel, correctly orientated. If that's not the case, you know you have to fix the inertia in the code.

- Starting Gazebo
- Inertia Tags
- **Collision Tags**
- Robot Simulation in Gazebo
- Gazebo Communication

- Gazebo needs more than the visual and inertial tags to simulate the robot. You will have to add <collision> tags to the links so that Gazebo can compute how two parts collide with each other.
- **Without the collision tags**, the robot will fall through the ground and continue falling indefinitely.
- **With a collision property**, the robot will collide with the ground and, thus, not fall.

- On top of that, that property will be used to **compute collisions** between different parts of the robot, or collisions with other robots and elements in an environment.
- For example, if the robot collides with a wall or an object, it will bump into it and not go through it. Once again, the <visual> tag doesn't do any of this.

- A <collision> tag will contain more or less the same thing as a <visual> tag: <geometry> and <origin> tags. You will basically define a shape.
- As a general rule, **for a collision, you will use a simpler shape** than for the visual (if possible).
- The reason is simple—the more complex the shape, the **more computation power** will be required to compute the collision between the link and other elements. This could slow down the simulation a lot. Thus, designing simpler shapes is a best practice.

- If you are using complex Collada files (a few MB) for visuals, **use simpler Collada or even STL files** for the collisions. You can add those files to the meshes folder and include them in the URDF.
- If the shape is close to a basic one (a box, cylinder, or sphere), then you can use a basic shape for the collision.
- For example, if you design a mobile robot and the base of the robot looks like a box, then you can use a complex Collada or STL file for the visual, only using a box for the collision. For a wheel, you can use a cylinder, sphere, and so on.

- You will add `<collision>` tags inside the `<link>` tags, in the `mobile_base.xacro` file.
- These tags should be at the same level as the `<visual>` and `<inertial>` tags, not inside them. The order doesn't really matter.
- Here is the `<collision>` tag for **base_link**:

`<collision>`

`<geometry>`

`<box size="{base_length} {base_width} {base_height}" />`

`</geometry>`

`<origin xyz="0 0 {base_height / 2.0}" rpy="0 0 0" />`

`</collision>`

- Let's add a **sphere collision** for our wheels, inside the wheel_link macro:

```
<collision>
```

```
<geometry>
```

```
<sphere radius="${wheel_radius}" />
```

```
</geometry>
```

```
<origin xyz="0 0 0" rpy="0 0 0" />
```

```
</collision>
```

- We just use the wheel radius property; there is no need for the wheel length here. And since it's a sphere, there's no need to add any rotation for the origin.

- Finally, we add the **collision for the caster wheel**. As stated before, we could simplify the sphere with a box, but as the caster wheel is in direct contact with the ground, we don't want to add unwanted friction.
- Thus, we use the same sphere for the visual and the collision:

<collision>

<geometry>

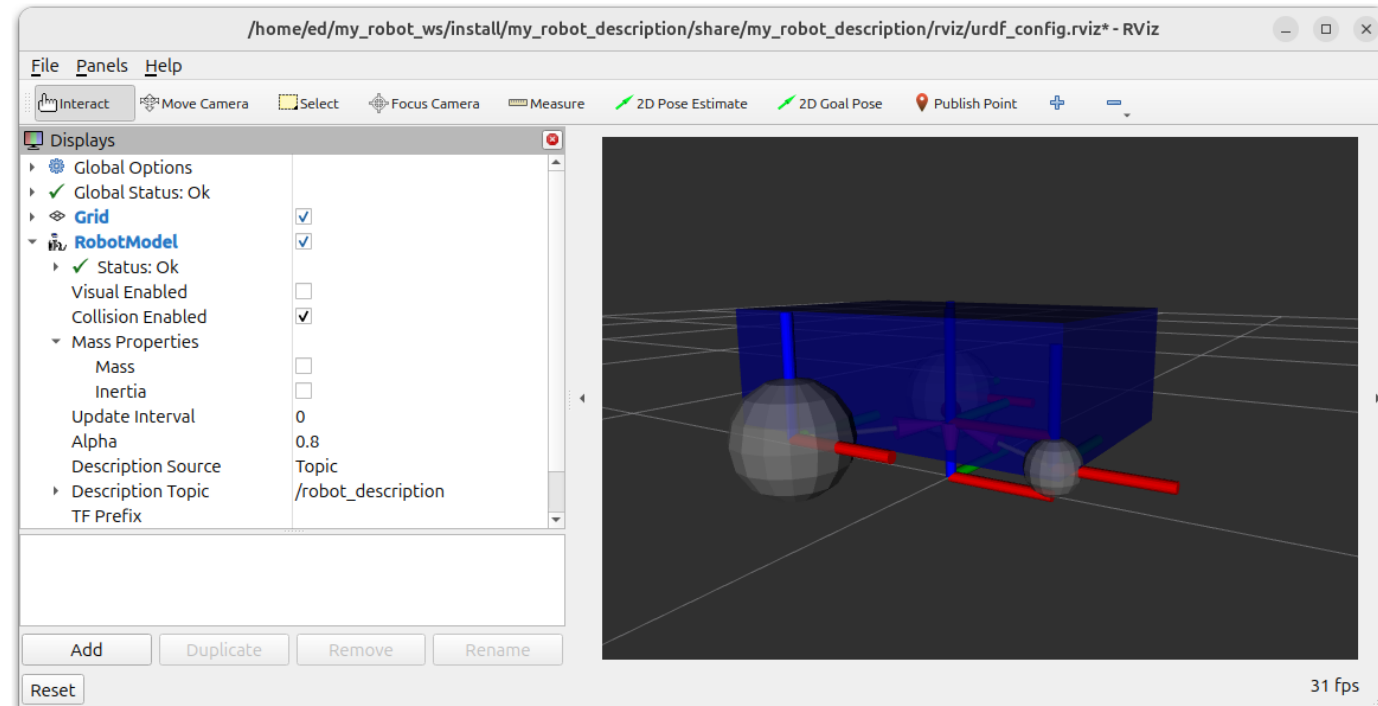
<sphere radius="{wheel_radius / 2.0}" />

</geometry>

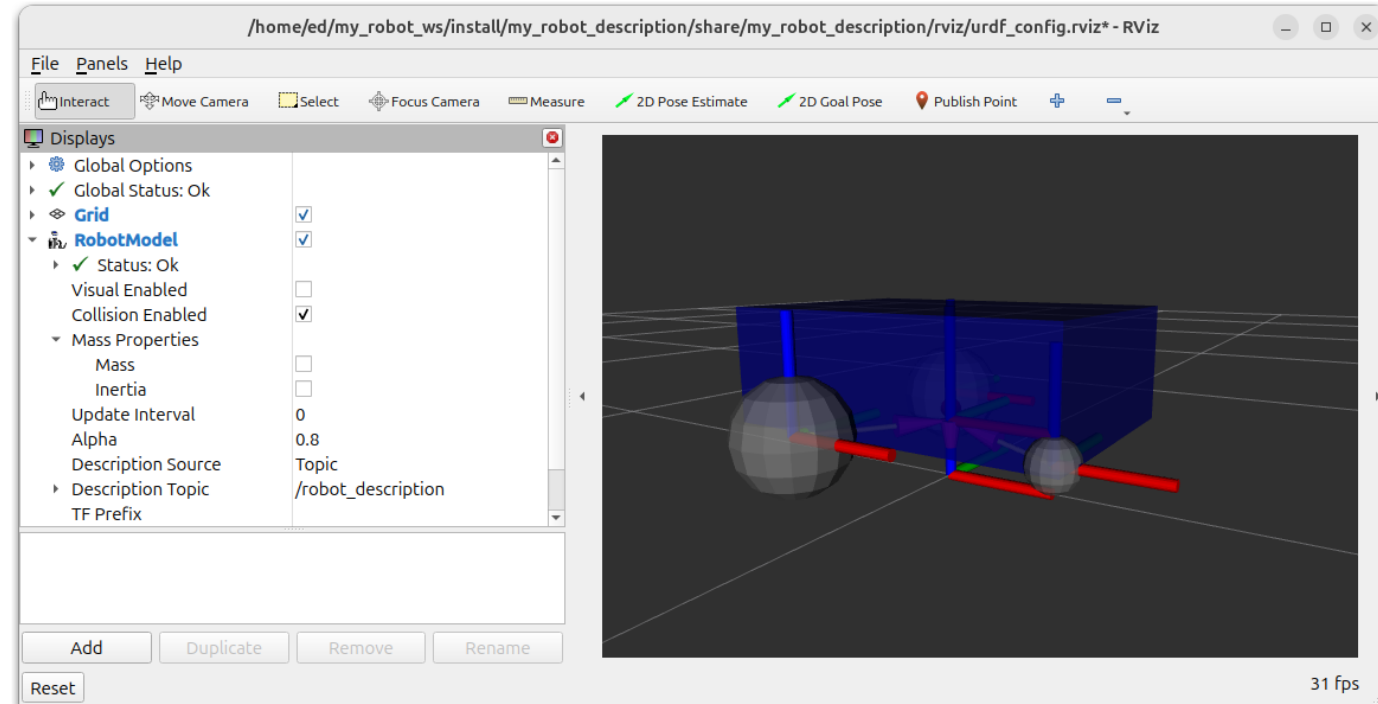
<origin xyz="0 0 0" rpy="0 0 0" />

</collision>

- As with everything we have written in the URDF, you can also visualize the collision properties in RViz.
- Build your workspace, source the environment, and start the display.launch.xml launch file again.**
- Open the RobotModel menu and **uncheck the Visual Enabled** box. Then, **check the Collision Enabled** box.



- There, you can see whether the collision properties are correct.
- In the Links menu (inside RobotModel), you can enable only some links if necessary, giving you a more precise view.
- If you see that some collision elements are not correctly placed, or if they are too big or too small, you can then go back to your URDF and fix them.



- Starting Gazebo
- Inertia Tags
- Collision Tags
- **Robot Simulation in Gazebo**
- Gazebo Communication

- Now that the URDF is done (and has been verified in RViz), we can spawn the robot in Gazebo.
- Here is what we will do:
 1. Run the robot_state_publisher node with the URDF as a parameter.
 2. Start the Gazebo simulator.
 3. Spawn the robot in Gazebo.
- Let's start this with the terminal commands.

- As usual, before creating a launch file, we will run each command one by one in different terminals, enabling us to clearly understand what we need to run, with all the necessary details.
- Open three terminals to start all the commands.
- The first thing to start is the `robot_state_publisher` node.

- In Terminal 1, run the following command:

```
ros2 run robot_state_publisher  
robot_state_publisher --ros-args -p  
robot_description:="$(xacro  
/home/<user>/ros2_ws/src/my_robot_description/  
urdf/my_robot.urdf.xacro)"
```

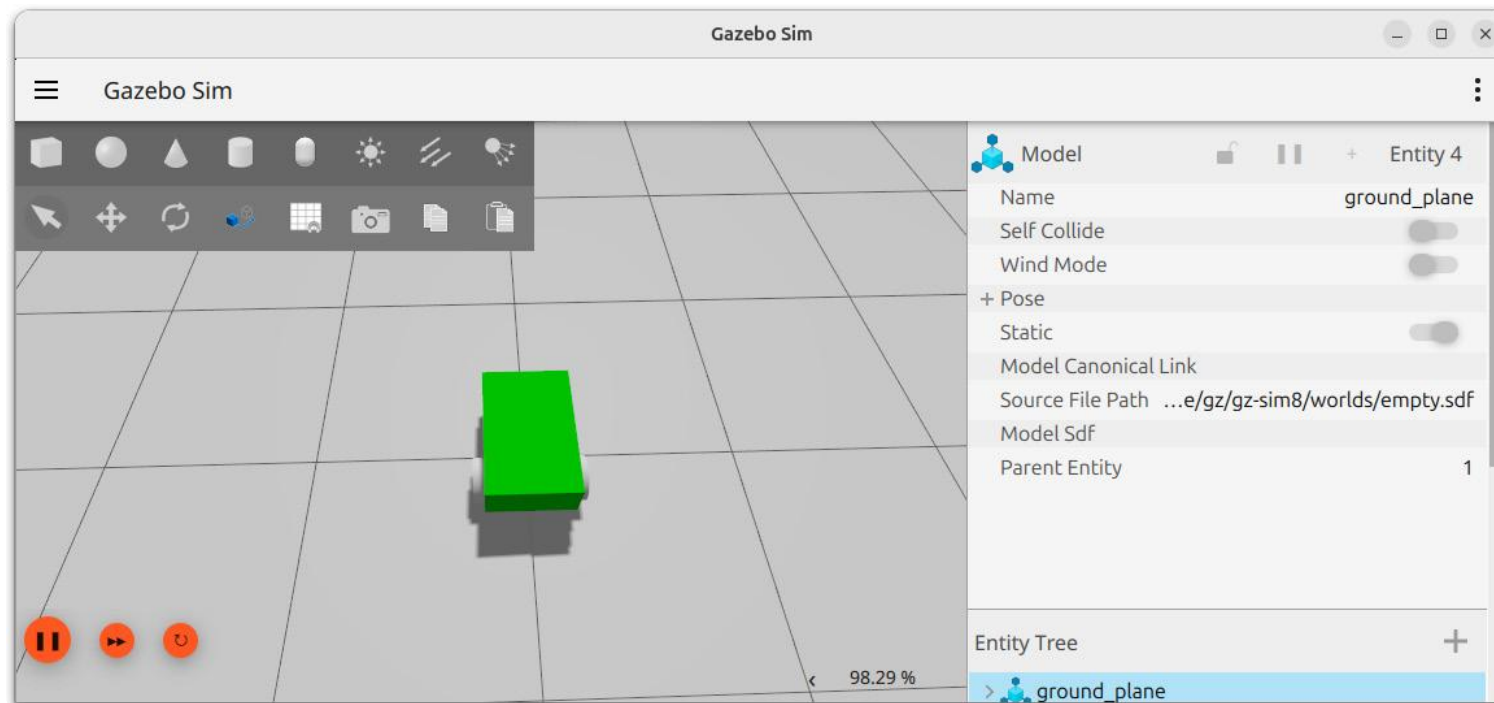
- After executing this command, the **robot_state_publisher node starts** and does three things—subscribes to /joint_states, publishes on /tf, and also publishes the URDF on /robot_description.

- In a second terminal, we **start Gazebo**. In fact, you could start Gazebo before robot_state_publisher; the order doesn't matter for those first two steps.
- In Terminal 2, run the following command:

```
ros2 launch ros_gz_sim gz_sim.launch.py  
gz_args:="empty.sdf -r"
```
- With this, we start an empty world in Gazebo. By default, the time is stopped when you launch Gazebo. We add the -r option to **start the time directly** so that we don't have to click the play button.

- Finally, once you have done the first two steps, you can spawn the robot in Gazebo.
- For this, we will use the **create executable from the ros_gz_sim package**. This will spawn a robot in Gazebo, using the URDF of the robot, which we can pass as a topic with the -topic option.
- As the robot_state_publisher node publishes the URDF on the robot_description topic, we can use this topic.

- In Terminal 3, run the following command:
ros2 run ros_gz_sim create -topic robot_description
- After running this command, you should see the robot in Gazebo:



- We are now going to write a launch file to start those three commands. This will be a good base on which we can add more things later on.
- Let's start by creating a new package in our workspace.
- We will start with the name of the robot or application, adding the `_bringup` suffix. Thus, we will create the `my_robot_bringup` package.

- Let's create this package, remove the unnecessary folders, and add a launch folder.
- We will also add a config folder, which we will use later:

```
$ cd ~/ros2_ws/src/
```

```
$ ros2 pkg create my_robot_bringup --build-type  
ament_cmake
```

```
$ cd my_robot_bringup/
```

```
$ rm -r include/ src/
```

```
$ mkdir launch config
```

- Now, in the CMakeLists.txt file of the my_robot_bringup package, add the instruction to install the launch and config folders:

```
install(  
  DIRECTORY launch config  
  DESTINATION share/${PROJECT_NAME}/  
)
```

- First, create a new file inside the launch folder. As this launch file will be the main one, let's simply use the name of the robot (or robotics application), my_robot.launch.xml:

```
$ cd ~/ros2_ws/src/my_robot_bringup/launch/
```

```
$ touch my_robot.launch.xml
```

- Open the file and write the minimal code for an XML launch file:

```
<launch>
```

```
</launch>
```

- Then, inside this `<launch>` tag, let's add everything we need, step by step.
- The beginning of the launch file will be very similar to the `display.launch.xml` file that we wrote earlier, so we can basically copy and paste a few parts.
- We start by adding a variable for the path to the URDF file:

```
<let name="urdf_path" value="$(find-pkg-share  
my_robot_description)/urdf/my_robot.urdf.xacro"  
>
```

- Now, we can start the robot_state_publisher node:

```
<node pkg="robot_state_publisher"  
exec="robot_state_publisher">  
  
<param name="robot_description"  
value="$(command 'xacro $(var urdf_path)')" />  
</node>
```

- Then, we start Gazebo with an empty world, and we also use the -r option to start the time automatically:

```
<include file="$(find-pkg-share  
ros_gz_sim)/launch/gz_sim.launch.py">  
  
<arg name="gz_args" value="empty.sdf -r" />  
</include>
```


- Finally, we spawn the robot in Gazebo:

```
<node pkg="ros_gz_sim" exec="create" args="-topic  
robot_description" />
```

- That's it for the launch file for now.
- Later, we will add more things and also start RViz to visualize the TFs. For now, we just want to see the robot in Gazebo.

- There is one thing we can do to make things a bit cleaner—as we are using files, nodes, and launch files from other packages, let's add a dependency to them in the package.xml file of the my_robot_bringup package.
- After the **<buildtool_depend>** line, add these lines:
<exec_depend>my_robot_description</exec_depend>
<exec_depend>robot_state_publisher</exec_depend>
<exec_depend>ros_gz_sim</exec_depend>

- We use the less strict `<exec_depend>` tag instead of `<depend>`, as we **only require those dependencies to run** the launch file, not to compile any code.
- With this, if, for example, you didn't install the `ros_gz_sim` package and you try to build the `my_robot_bringup` package, you will get an error when running `colcon build`, and then you can fix things right away.
- Without those lines, the build would work, but you would then get an error when starting the launch file, which can be a big problem, especially in a production environment. So, the best practice is to specify all the dependencies you need in the `package.xml` file.

- Now, save all the files, build the workspace, source the environment, and start the launch file (make sure that Gazebo is not running in another terminal before you do this):

```
$ ros2 launch my_robot_bringup  
my_robot.launch.xml
```

- You should get the same result as when we ran all three commands in the terminal.

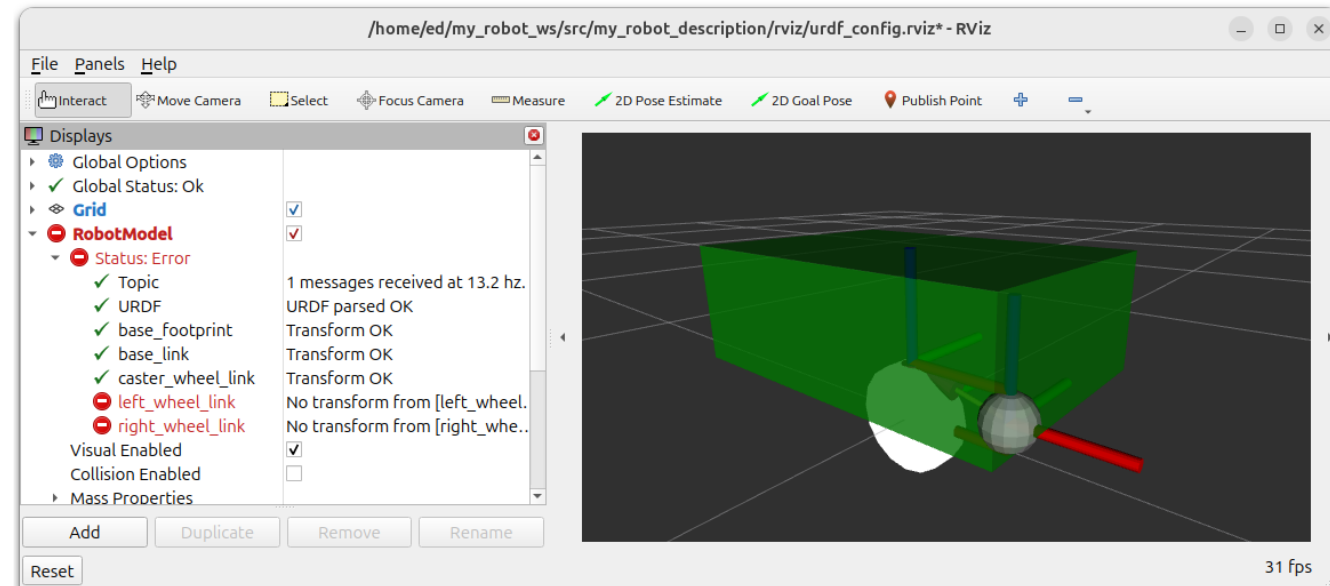
- Our mobile robot is now simulated in Gazebo with physics properties.
- Now what? **The robot is not doing anything.** We need to add control plugins so that we can simulate the hardware of the robot and do the following:
 1. Send commands to make the robot move in Gazebo, just as if it were in the real world.
 2. Read all necessary joint states from the robot to get all the TFs in our ROS 2 application.
- Before we start discussing Gazebo systems and bridges, let's dive a little bit deeper and understand what's missing and what we need to add.

- When you start the `my_robot.launch.xml` launch file, you see the robot in Gazebo. However, **we don't have any way to control it.**
- In a terminal, if you list all nodes, topics, services, or even actions, you won't find anything we can use.
- Also, after starting the launch file, if you print the TF tree, you won't see the TF for the right or left wheel.

- You can observe the same thing with RViz—to make things simple, you can start RViz using the previous configuration we saved:

```
ros2 run rviz2 rviz2 -d
~/ros2_ws/src/my_robot_description/rviz/urdf_config.rviz
```

- You should see some errors in RobotModel, saying **No transform from [left_wheel_link]** and **No transform from [right_wheel_link]**.



- This lack of TF is because **nobody is publishing on the /joint_states topic**. So, what do we need to do?
- **For a real robot**, you would **create a hardware driver** to be able to control the wheels. This driver would expose a topic/service/action interface so that you could make the robot move.
- Then, you would read position/velocity data from encoders and publish this data on the /joint_states topic. With this, the loop is closed.

- **For a Gazebo simulation**, we will do the same thing but, of course, without hardware.
- We will **use Gazebo plugins (systems)** to simulate the control of the robot and get the joint states.
- Then, we will configure a bridge to make those plugins communicate with our ROS 2 application.
- Let's start with the Gazebo systems.

- A Gazebo system is basically the simulation of a **hardware component**. You could have a system simulating a camera and publishing images, another one monitoring a battery state, and so on.
- For now, we will use two systems—one to control a differential drive robot (two parallel wheels), and one to publish the joint states.
- Now, the good news is that **there are a lot of existing systems already available** to use, including the two that we need.

- The bad news is that **documentation** for those systems is **almost non-existent**, and you will have to dive into the code itself to find what to include in your own code.
- For Gazebo Harmonic and ROS 2 Jazzy, you can find all available Gazebo systems here on GitHub:
<https://github.com/gazebo/gz-sim/tree/gz-sim8/src/systems>
- Now, where are we going to **add our systems** for the robot we want to simulate? We will do this **in the URDF**.

- The Gazebo systems for our robot will be specified in the URDF. So, we need to return to the my_robot_description package.
- Our URDF is now split into three files: one with common properties, one with the description of the robot (links and joints), and one to include the other two.
- To add the Gazebo systems, we will **create yet another Xacro file, dedicated to all Gazebo-related stuff.**
- By separating this file from the other ones, we make things cleaner. If, later on, you want to use the URDF without Gazebo, you only need to remove the inclusion of the Gazebo file.

- In the urdf folder of your my_robot_description package, add a fourth file, named **mobile_base_gazebo.xacro**.

- Open the file and add the minimal Xacro code:

```
<?xml version="1.0"?>
```

```
<robot  
  xmlns:xacro="http://www.ros.org/wiki/xacro">  
  </robot>
```

- Now, in **my_robot.urdf.xacro**, include the file after the two other ones:

```
<xacro:include filename="$(find  
my_robot_description)/urdf/mobile_base_gazebo.xacro" />
```

- The Xacro file is ready, and we can now add the systems.

- The first system we will add is a **differential drive controller**. By differential drive, we mean a robot controlled by two wheels, one on each side of the robot.
- If you browse the available systems, you can find a **diff_drive folder**—usually in ROS, we use diff drive as an abbreviation of differential drive.
- In this folder, you will see a **DiffDrive.hh** file. Open this file, and there, near the beginning, you will find the XML tags related to the system (it's possible that some tags will be missing here; for some systems, you might have to read the complete source code to find all available tags).

- Here is how to add the system to our Xacro file (**mobile_base_gazebo.xacro**):

```
<gazebo>
```

```
<plugin
```

```
  filename="gz-sim-diff-drive-system"
```

```
  name="gz::sim::systems::DiffDrive">
```

```
  <left_joint>base_left_wheel_joint</left_joint>
```

```
  <right_joint>base_right_wheel_joint</right_joint>
```

```
  <frame_id>odom</frame_id>
```

```
  <child_frame_id>base_footprint</child_frame_id>
```

```
  <wheel_separation>0.45</wheel_separation>
```

```
  <wheel_radius>0.1</wheel_radius>
```

```
</plugin>
```

```
</gazebo>
```


- We start with a `<gazebo>` tag. Everything related to Gazebo will be in such tags.
- Then, we include the system with a `<plugin>` tag. We also need to specify the filename and name for the system.

- Here is a bit more information about the different parameters for this diff drive system:
1. **left_joint and right_joint:** You need to provide the exact name of the joints you have defined for the wheels in the URDF.
 2. **frame_id:** As the robot moves, we will keep track of where it is relative to its starting position. This starting position will be called odom (short for odometry).
 3. **child_frame_id:** We write base_footprint, as it is the root link for our robot and the one we want to use for odometry tracking.

4. **wheel_separation:** We can compute that from the URDF. The base width is 0.4, and the origin for each wheel is centered on the wheel. As each wheel length is 0.05, we need to add $0.4 + 0.025 + 0.025$, which makes 0.45.
5. **wheel_radius:** We get this value from the URDF, which is defined as 0.1.
6. **Acceleration and velocity min and max:** Optionally, you can set some limits. This can be a good idea so that the controller doesn't accept a command that would make the robot move too fast and, potentially, become dangerous to itself or an environment. For the values, once again, you should use the metric system and radians for angles.

- That's it for the diff drive system. Now, in addition, there is one setting we need to add for the caster wheel.
- If you remember, the caster wheel is a passive joint, so we defined it as a fixed sphere.
- As the wheels turn and the robot moves, there will be some friction between the ground and the caster wheel.
- **You won't see the friction that much in Gazebo**, but it will slow down the robot a bit, and later on, if you visualize the robot in RViz, you won't have the same result.

- So, we will **reduce the friction for the caster wheel**. You can add this code **just before the code for the diff drive system**:

```
<gazebo reference="caster_wheel_link">  
  <mu1 value="0.1" />  
  <mu2 value="0.1" />  
</gazebo>
```

- There are two parameters, mu1 and mu2, that you can set to have more control over the friction. We have chosen the value 0.1 here. Later, you could reduce this value even more if needed.

- We have added a system to control the wheels, but before we test it, let's finish the Xacro file and **add the second system** we need.
- The diff drive system alone won't publish the joint states for the wheels; we need to add a joint state publisher system.
- Go back to the systems page on GitHub, and you will find a **joint_state_publisher folder**. In this folder, you can get the documentation for the XML tags in the **JointStatePublisher.hh** file.

- Let's add the system to the Xacro file, after the previous one:

```
<gazebo>
```

```
<plugin
```

```
  filename="gz-sim-joint-state-publisher-system"
```

```
  name="gz::sim::systems::JointStatePublisher">
```

```
</plugin>
```

```
</gazebo>
```

- The joint state publisher system is easier to set up. Also, we don't specify any `<joint_name>` tag here to publish all available joint states.
- If your robotics system contains a lot of joints, it could be useful to only specify the joints you want to use.
- Our `mobile_base_gazebo.xacro` file is now complete, and we won't need to modify anything else in the URDF. We can spawn the robot in Gazebo again and see how it interacts with those systems.

- Starting Gazebo
- Inertia Tags
- Collision Tags
- Robot Simulation in Gazebo
- **Gazebo Communication**

- Gazebo uses topics and services, but those are independent from ROS 2.
- Thus, the systems we have just added will work, but they will only have a Gazebo interface.
- You can verify this by starting the **my_robot.launch.xml** file again—**make sure to compile and source the workspace** beforehand so that you get the updated URDF.

- Then, in another terminal, list all Gazebo topics.
- The list will contain quite a lot of things; here, I have only included the ones that we will use:

gz topic -l

/model/my_robot/tf

/world/empty/model/my_robot/joint_state

/model/my_robot/cmd_vel

- The first topic that ends with /tf will contain the TF from the odom frame to base_footprint.
- The one with /joint_state will contain the joint states for both wheels, and the topic with /cmd_vel will be used to send a velocity command to the robot.
- However, **if you check the ROS 2 topics with ros2 topic list, you won't see the /cmd_vel topic.** You will have /joint_states and /tf, but only because the robot_state_publisher node creates a subscriber and publisher for those topics.
- **Nothing is published;** you can verify this with ros2 topic echo <topic>.

- Thus, **from the ROS 2 side, we can't communicate with Gazebo.**
- We will need to **create a bridge** between ROS 2 and Gazebo using the `ros_gz_bridge` package.
- To do that, we will run the `parameter_bridge` node from the `ros_gz_bridge` package, with a configuration for the interfaces that we want to bridge.

- Let's start with the configuration file. In the **_robot_bringup package**, create a config folder .

```
$ cd ~/ros2_ws/src/my_robot_bringup/
```

```
$ mkdir config
```

- Open the CMakeLists.txt file and add the config folder in the dependencies:

```
find_package(ament_cmake REQUIRED)
```

```
install(
```

```
  DIRECTORY launch config
```

```
  DESTINATION share/${PROJECT_NAME}/
```

```
)
```

```
ament_package()
```

- Create a new file named **gazebo_bridge.yaml** inside the config folder.

```
$ cd config
```

```
$ touch gazebo_bridge.yaml
```

- Open this file to write the configuration. Here is the first bridge we will create:

```
- ros_topic_name: "/cmd_vel"
```

```
  gz_topic_name: "/model/my_robot/cmd_vel"
```

```
  ros_type_name: "geometry_msgs/msg/Twist"
```

```
  gz_type_name: "gz.msgs.Twist"
```

```
  direction: ROS_TO_GZ
```

- Here are the different fields that we will use:
- **ros_topic_name:** The topic name on the ROS 2 side. Either you choose the topic name (/cmd_vel doesn't exist yet, so we create it) or you make it match with an existing one (for the next one, we will have to specify exactly /joint_states).
- **gz_topic_name:** The topic name on the Gazebo side. We found it with `gz topic -l`.
- **ros_type_name:** The topic interface for ROS 2.

- **gz_type_name:** The topic interface for Gazebo. You can find it with `gz topic -i -t <topic>`.
- **direction:** Either `ROS_TO_GZ`, `GZ_TO_ROS`, or `BIDIRECTIONAL`.
- For example, `/cmd_vel` is a topic that we publish in ROS 2 and subscribe in Gazebo, so we use `ROS_TO_GZ`.
- For `/joint_states`, we publish in Gazebo and subscribe in ROS 2, so that will be `GZ_TO_ROS`.
- You can use `BIDIRECTIONAL` if you want to have publishers and subscribers on both sides of the same topic.

- As you can see, we need to provide the topic name and interface on both sides and specify which direction to use for the communication.
- With this, the `ros_gz_bridge` will create the connection.
- With this first bridge, we will be able to send commands to the robot to make it move with the diff drive system.

- Let's now add the configuration for the `/joint_states` topic (published by the joint state publisher system):

- **ros_topic_name:** `"/joint_states"`

gz_topic_name:

`"/world/empty/model/my_robot/joint_state"`

ros_type_name: `"sensor_msgs/msg/JointState"`

gz_type_name: `"gz.msgs.Model"`

direction: `GZ_TO_ROS`

- That will allow us to get all joint states for the robot and, thus, see the wheel TFs in RViz.

- Finally, to get the odom to base_footprint TF (published by the diff drive system), we also add this bridge:
 - **ros_topic_name: "/tf"**
 - gz_topic_name: "/model/my_robot/tf"**
 - ros_type_name: "tf2_msgs/msg/TFMessage"**
 - gz_type_name: "gz.msgs.Pose_V"**
 - direction: GZ_TO_ROS**
- The configuration file is complete. As we have already added the instruction to install it in CMakeLists.txt, there is no need to do anything else.

- We can now add a new node to our **my_robot.launch.xml** file to start the bridge, using the YAML configuration file we've just created.
- First, at the beginning of the file, let's add a new variable to find the path for the configuration file:

```
<let name="gazebo_config_path" value="$(find-pkg-share  
my_robot_bringup)/config/gazebo_bridge.yaml" />
```

- Then, after you spawn the robot in Gazebo with the create executable from ros_gz_sim, start the Gazebo bridge.
- You will need to pass the configuration file inside a config_file parameter:

```
<node pkg="ros_gz_bridge" exec="parameter_bridge">  
  <param name="config_file"  
    value="$(var gazebo_config_path)" />  
</node>
```

- As we use the `ros_gz_bridge` package inside `my_robot_description`, we will also add a new dependency inside the **package.xml** file:

```
<exec_depend>ros_gz_bridge</exec_depend>
```

- The Gazebo bridge is now correctly configured.
- When you start your application, ROS 2 and Gazebo will be able to communicate with each other.

- In this final section, we will make sure that everything works by testing the behavior of the robot, and also by visualizing the robot and TFs in RViz.
- Save all files, build and source the workspace, and **start the `my_robot.launch.xml` file again.**

```
$ ros2 launch my_robot_bringup my_robot.launch.xml
```

- In another terminal, list all the topics, and you will see the `/cmd_vel` topic that we configured previously.

- Send a velocity command from the terminal:

```
$ ros2 topic pub /cmd_vel geometry_msgs/msg/Twist  
"{linear: {x: 0.5}}"
```

- The robot should start moving in Gazebo (to stop, send the same command with {x: 0.0}).
- If you see the robot moving, it means that the bridge is correctly configured, as the ROS 2 topic can reach the Gazebo system. It also means that the diff drive system works.

- To achieve a better way to control the robot and make more tests, you can run this node instead:

```
$ ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

- This will listen to your keyboard and publish to the /cmd_vel topic (if you use a different name for the topic, simply add a remapping with --ros-args -r).
- Now, we have validated that the robot can move in Gazebo when we send a command.

- To check the TFs, you can do the following:
 1. Subscribe to the `/joint_states` topic and see the states for both the right and left wheels.
 2. Subscribe to the `/tf` topic and see all published TFs.
 3. Print the TF tree (`ros2 run tf2_tools view_frames`), which should contain all the TFs, including the two TFs for the wheels and an additional one between the `odom` and `base_footprint`.

- So, we can control the robot and get the correct TFs in our ROS 2 application.
- Finally, let's start RViz. You can use the command line, but you can also add RViz directly inside the launch file if you want to.
- In this case, we will first create a variable to find the RViz config path:

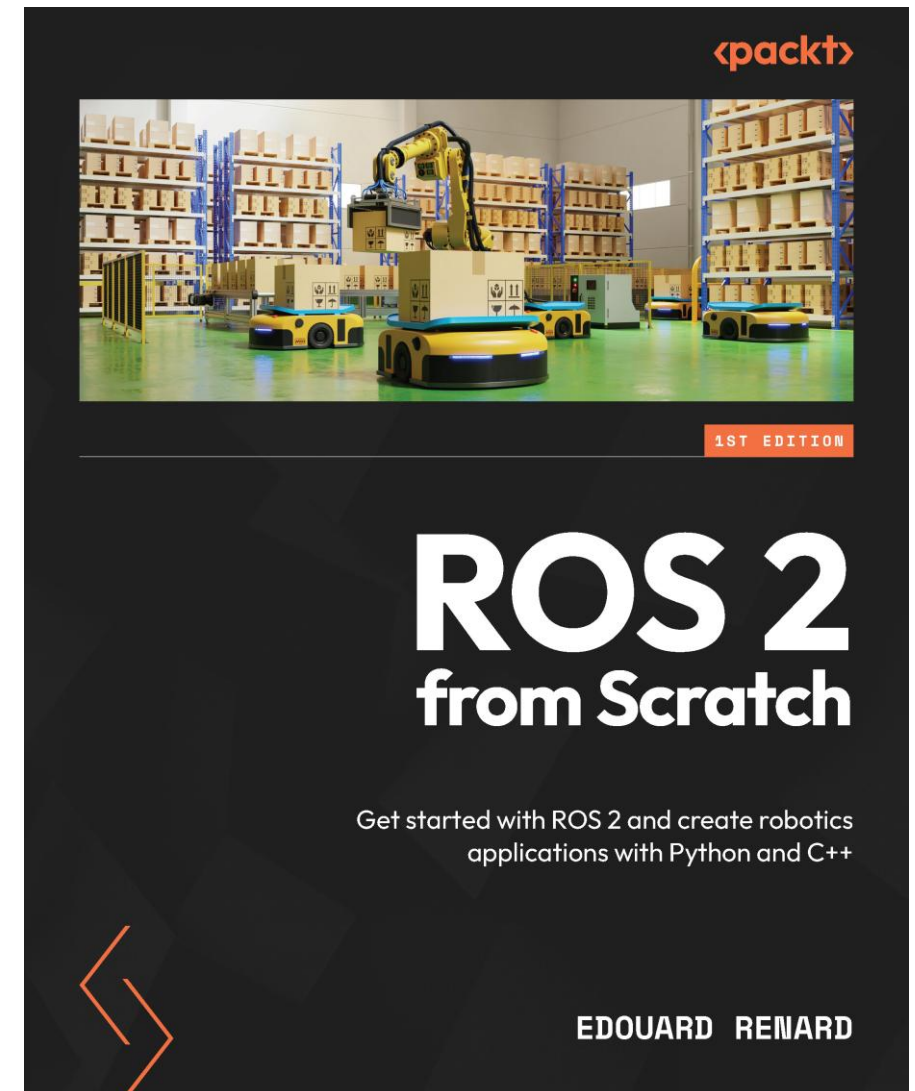
```
<let name="rviz_config_path" value="$(find-pkg-share  
my_robot_description)/rviz/urdf_config.rviz" />
```

- We will use the file we previously created in my_robot_description.
- You could also create a new RViz configuration file and install it in my_robot_bringup.
- Then, we start RViz after all the other nodes:

```
<node pkg="rviz2" exec="rviz2" args="-d $(var  
rviz_config_path)" />
```

- With this, when you start the launch file, you will have both Gazebo and RViz.
- The TF errors that we previously got in RViz should not be there anymore.
- One thing you can do is to select odom as the fixed frame, inside Global Options. With this setting, when the robot moves in Gazebo, you will also see it moving from its starting position in RViz.
- Our application is now finished. The Gazebo systems correctly work and can communicate with the ROS 2 side. The loop is closed.

- ROS 2 from Scratch by Edouard Renard
- Chapter 13



Thank you for your attention.

Any Questions?

